

Lecture 6: Media Device Software Platform — HAL and PAL

Lecture 6 continues the transition from the conceptual media-device architecture toward concrete software implementation. In Lecture 5, the complete host software stack was introduced: applications, middleware, platform, SDKs and hardware abstraction. This lecture zooms further into the platform layer and explains how HAL and PAL cooperate in a real media device.

As in the previous lectures, we start from the broad picture and gradually move into more concrete details. The overall device architecture still stands: production, distribution and consumption remain the main media-system stages, and the consumer device still consists of reception, processing and presentation blocks. What changes now is the level of detail. Instead of looking only at blocks and signal paths, we now examine how those blocks are exposed through platform APIs and how they become programmable objects.

The basic behavior of hardware blocks within HAL was already covered earlier and practiced in Project 1. Concepts such as demultiplexers, PID filtering, PSI/SI parsing, transport streams and signal paths should already be familiar. Similarly, the operating system and common libraries are not examined in depth here, because they are general platform topics students have likely already seen elsewhere.

The key focus of this lecture is the DTV SDK, as an important part of the Platform Abstraction Layer. The DTV SDK is not the entire platform, but it is one of the most important platform components for digital TV software. It is the layer where broadcast-media concepts become usable software abstractions.

DTV SDK as a Platform Abstraction

To make the discussion concrete, the lecture uses Broadcom Nexus as an example of a DTV SDK. The term DTV SDK is generic; Nexus is one particular implementation for Broadcom-based systems.

Broadcom also has a lower-level part of the stack called Magnum. Magnum can be understood as a lower hardware-facing layer, but it is not the layer typically used by application or middleware developers. From the perspective of host software, Magnum is hidden underneath Nexus. Programmers communicate with Nexus, and Nexus becomes the relevant platform API.

Nexus is organized into modules and interfaces. This already reveals its abstraction level. The modules are not named only after physical hardware blocks. They are named according to functional groups and according to what the programmer wants to achieve.

For example, the Transport module is not just one hardware block. It represents signal-path and data-routing functionality. Even the word signal is already an abstraction here. In hardware, a signal may refer to an electrical waveform. In media software, signal means content data moving through a sequence of processing phases. Nexus therefore exposes transport as a way to construct and manage media data paths.

A poor SDK would act only as a thin tunnel to hardware. It would expose hardware almost directly and add little useful structure. Quality platforms such as Nexus are richer than that. They provide objects and interfaces that help construct processing paths in software.

Frontend, Transport and Display as Abstractions

The Frontend module provides another example of abstraction. In the simplest broadcast case, frontend maps to tuner and demodulator functionality. Conceptually, however, frontend means a source-facing block that receives media signal from somewhere and delivers it further into the processing pipeline.

In a broader sense, a file input or IP input could also be seen as a type of frontend, depending on the SDK or system design. Nexus has its concrete implementation, but the term frontend is already broader than one physical tuner.

The Display module is similar. Of course, display hardware exists somewhere in the device, but display in the platform sense means presentation control. It includes how video is positioned, scaled, routed to outputs and placed into windows. A video window is not a physical hardware object. It is a platform abstraction used to describe where decoded video should appear.

These examples show how PAL differs from raw HAL. HAL exposes hardware capabilities. PAL introduces software objects that correspond to useful developer concepts.

Nexus Transport Interfaces: InputBand, ParserBand and PidChannel

Three important Nexus transport interfaces are InputBand, ParserBand and PidChannel.

None of these exists as a simple physical hardware block. They are logical abstractions used to describe operations over a transport stream.

InputBand defines where the signal comes from. ParserBand represents a logical demux or parsing context operating over that input. PidChannel represents filtering or routing of a specific PID from the parser band toward a consumer.

For example, when an MPEG transport stream enters the system, the software first defines the input source through an InputBand. Then it defines a ParserBand that operates on that input. On top of the ParserBand, the software opens PID channels, each configured for a specific PID. These PID channels can then be routed toward consumers such as a video decoder, audio decoder, message filter or recorder.

This is much richer than commanding hardware devices. The programmer is creating objects, configuring them and connecting them into a media pipeline.

The same idea continues with VideoDecoder and Display. A video decoder consumes compressed video data coming from a PID channel. Its output can be routed to a video window, and that video window belongs to a display. The display then manages final presentation and output routing.

The basic logic of Nexus is therefore clear: create objects, configure them, connect them and form processing paths. Each processing path corresponds to a real media use case.

Processing Pipelines as Software Objects

The Nexus approach is not unique to digital TV SDKs. Many SDKs that deal with processing pipelines follow a similar model. Each processing phase is represented as an object. Each object has inputs, outputs, settings and lifecycle operations. By connecting these objects, software builds a processing chain that corresponds to a use case.

When middleware wants to start recording, switch service, play content from IP, play content from a tuner, or show a mosaic channel list with multiple small preview videos, each use case corresponds to a different configuration of SDK objects.

The task of middleware is to know the SDK well enough to reconfigure the system correctly. For each use case, middleware may contain a function that builds the required pipeline from Nexus objects.

Nexus documentation describes this idea as a filter graph: initialize the platform, open the needed interfaces, connect them into a graph, configure settings and start processing. This is the key transition from the signal-path diagrams of earlier lectures into real API calls.

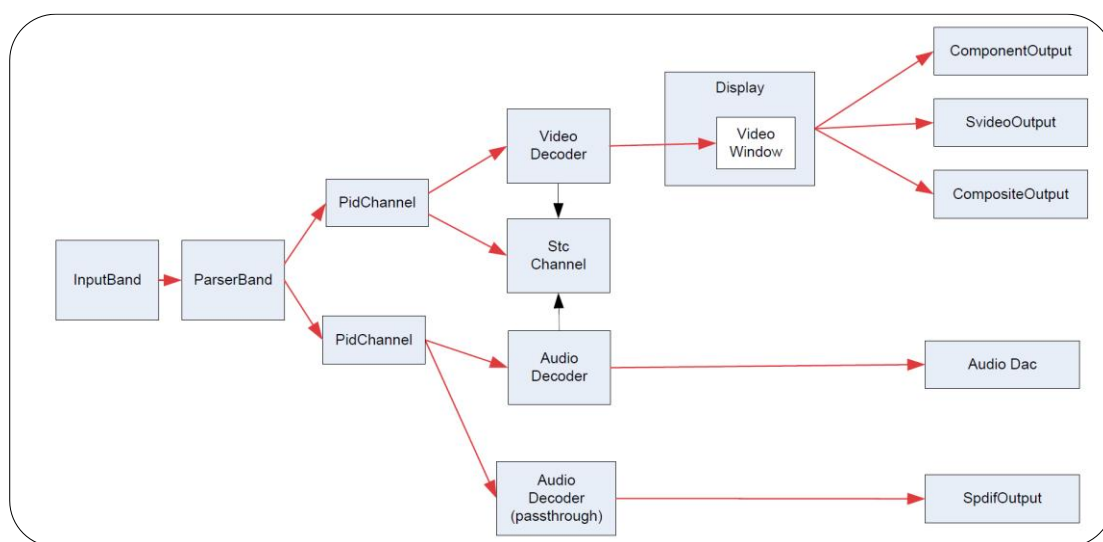


Figure 1. Nexus object/filter graph depicting a signal path at the platform level

Linux Platform Abstractions

The DTV SDK is central, but it is not the only component needed at the platform layer.

On Linux systems, one may encounter the Linux DVB API. The DVB API attempts to define a standard platform-level interface for digital TV functionality, exposing concepts such as frontend, demux, video and audio. However, it is relatively poor in abstraction compared with a richer SDK such as Nexus. In many cases, it feels closer to HAL than to a full domain-specific platform API.

Conditional access usually comes through separate libraries or SDKs. This is because CAS functionality is proprietary and security-sensitive. The CAS SDK must cooperate with the DTV SDK, but it is often delivered by a separate vendor.

A/V playback frameworks such as FFmpeg may also need to be integrated with transport and decoding functionality, especially for use cases involving files, IP streams, recording or broadcast playback.

If the upper layer is a web browser, native functionality must be exposed to JavaScript through plugins or extensions. The browser may see a JavaScript object with fields and methods, while those methods call native C or C++ functionality underneath. This separation between the application world and the native world is typical.

In Linux-plus-web systems, the upper layer may be JavaScript running inside a browser, while the native layer is C or C++. In Android, the upper layer is typically Java or Kotlin, while the native layer is C or C++ through the Android NDK. Android uses JNI as the bridge between these two worlds.

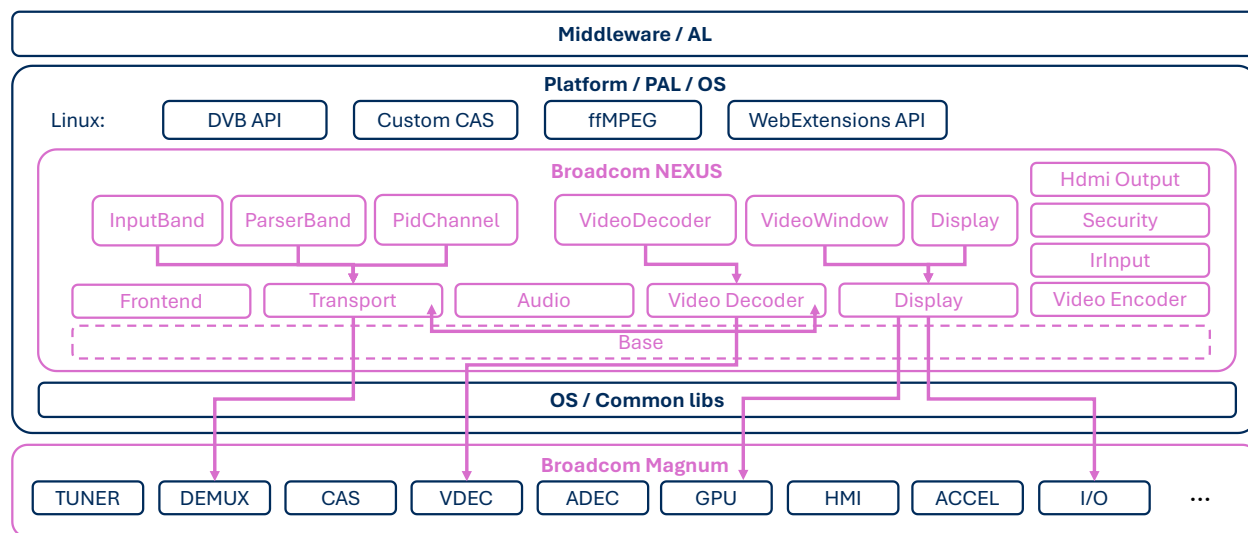


Figure 2. Platform stack within host software showing DTV SDK Nexus and other Linux-based modules

Android Platform Abstractions

Android defines its own platform abstractions for media and broadcast.

For broadcast functionality, Android provides the Tuner SDK. If a vendor wants to support broadcast reception under Android, the lower DTV SDK must be adapted or wrapped to match the Android Tuner SDK API. In this sense, Tuner SDK can be understood as a high-level Android wrapper around the vendor DTV SDK.

For conditional access, Android provides Media CAS. For video decoding and playback, Android provides MediaCodec. For audio output, Android commonly uses AudioTrack.

These abstractions are important for this course because they are used in practical work. Media CAS may not be used in detail, but Tuner SDK, MediaCodec, AudioTrack and JNI are central for connecting broadcast or media input to playback.

The Linux flavor may include DVB API, native plugins, browser extensions, FFmpeg and vendor SDKs such as Nexus. The Android flavor includes Tuner SDK, JNI, MediaCodec, AudioTrack and Media CAS. Both solve similar problems, but through different software ecosystems.

Layering Nexus with Android

If the Nexus-based platform picture is layered together with Android, the architecture becomes more complex.

The Android Tuner SDK exposes an API upward. Above it, in middleware, there are Android components such as the TV Input Framework and TV services. These will be discussed in more detail in the following lecture. For now, the important point is how they connect downward.

Middleware talks to the Tuner SDK through the Tuner Java API. The Java API is translated through JNI into the Tuner Native API. Java or Kotlin code lives in the upper Android world, while JNI bridges it to native C or C++ code underneath.

Android also introduces the Tuner Resource Manager, or TRM. TRM exists because Android is not designed as a single monolithic TV application. It is a multi-application system, where more than one client may request tuner resources.

Traditional DTV SDKs were not originally designed with this assumption. In classical digital TV software, a monolithic TV stack owns the tuner, demux, decoders and playback pipeline. In Android, multiple applications and services may coexist. If two applications try to use the tuner or configure the demux, the system needs a central authority to manage resources.

Nexus may provide some resource management internally, but Android cannot rely on every vendor SDK to behave the same way. Therefore, Android introduces its own

resource-management logic. This creates some duplication and explains why real systems often contain wrappers around wrappers.

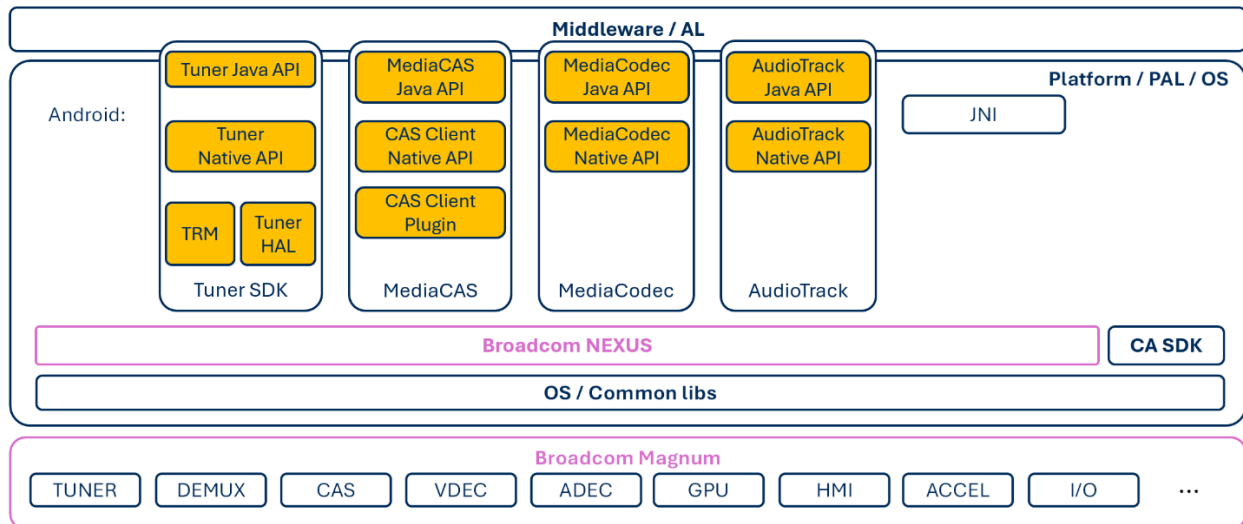


Figure 3. Android platform abstractions

Excursus: Why Wrappers Exist

Wrappers appear because software layers need controlled boundaries.

Imagine that functionality B depends on functionality A, but A was not written by the same team. The developers of B may not fully understand A, may not want to depend directly on its exact API, or may not even know which implementation of A will finally be used. A wrapper around A creates a cleaner local API that the B side can control.

This is especially important in co-design situations. The upper software may be developed while the lower platform is still changing, or before the final SoC is selected. Developers may start on a Broadcom platform and later be asked to support MStar or another vendor. If the software depends directly on Nexus everywhere, the port becomes difficult. If the software depends on a wrapper, the lower implementation can be replaced more easily.

Android's Tuner API is standard. Nexus is vendor-specific. Nexus may remain stable across Broadcom SoC families, but it will not help if the product moves to a different vendor SDK. A wrapper or standard abstraction is required.

Middleware vendors use the same idea. For example, iWedia Comedia middleware has its own abstraction layer toward lower platforms. This is sometimes called CHAL, or Comedia Hardware Abstraction Layer. Despite the name, it is not the SoC HAL discussed earlier. It is a middleware-facing adaptation layer that allows Comedia to run on different targets.

In modern Android-based systems, the chain may become long: vendor DTV SDK, Android Tuner API, middleware-vendor abstraction and finally middleware above that. These layers often reflect technical needs, historical evolution and business relationships.

Multiple wrappers may look redundant, and sometimes they only tunnel calls downward. However, refactoring a mature TV stack can be risky because behavior depends on many hidden assumptions. Wrappers are not always elegant, but they usually exist to standardize, reuse, port or scale software across different targets.

Android Platform Components

In Android, the standardized platform approach is to follow the Tuner HAL architecture. The vendor implements the HAL-facing contract, while the Android framework owns more of the upper behavior. This creates a cleaner separation between vendor-specific DTV implementation and Android middleware.

Media CAS follows a similar architectural idea for conditional access. Conditional access is proprietary and security-sensitive, so there is no universal implementation available by default. A Conditional Access SDK sits next to the DTV SDK, and the system needs a plugin or adaptation layer for each CAS vendor.

A product may support multiple CAS systems. Each CAS client may require its own plugin that talks to the corresponding CAS SDK. Such plugins may pass keys into a secure environment or TrustZone, where the CAS implementation can access them safely. These are the same service keys and control words discussed earlier, now viewed through the software-stack perspective.

MediaCodec is conceptually simpler. It provides Android's API for video decoding and playback. AudioTrack plays the corresponding role for audio playback. The practical challenge is connecting these Android APIs correctly to the underlying broadcast or media pipeline.

Example Platform Call Sequences

Architectural software block diagrams may not be enough to fully specify architecture and interactions within complex software. Therefore, as you will do also in Project 2, we need to express software behavior as UML sequence diagrams.

The goal is to show how a high-level middleware action becomes a sequence of platform calls, and how those calls eventually reach the vendor DTV SDK. The diagrams in this lecture are not intended to be perfect implementation blueprints, but they are accurate enough to demonstrate how the stack behaves.

Broadcast Tune Flow

The first example is broadcast tuning.

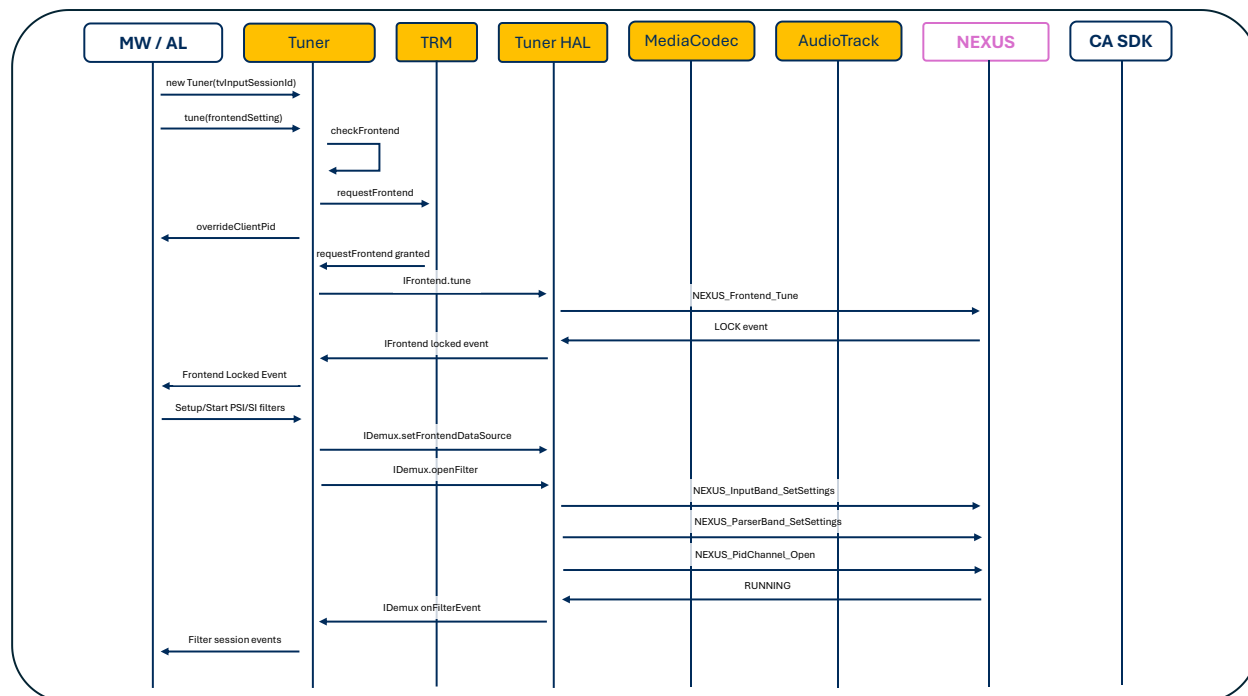


Figure 4. Broadcast tune UML sequence

At the top, middleware creates or obtains a Tuner object through the Android Tuner SDK API. In a TV application, this is associated with a TV input session. Middleware then calls a tune operation and provides frontend settings, such as frequency and broadcast-standard parameters.

Before the tune request reaches hardware, Android checks whether the needed frontend resource is available. This is handled by TRM. Once the frontend resource is granted, the request is forwarded to the Tuner HAL.

At the HAL boundary, the frontend is represented through an interface such as IFrontend, and the concrete operation is a tune call. Below this boundary, the vendor implementation translates the Android call into vendor SDK calls.

In the Broadcom/Nexus example, the call becomes a Nexus frontend tune operation. Nexus performs the tuning through the vendor platform implementation. When the tuner locks to the signal, a lock event propagates upward: from Nexus to Tuner HAL, from Tuner HAL to the Android Tuner API and finally to middleware as a frontend locked event.

This is a relatively direct tunneled operation. Middleware calls tune, Android manages the resource, HAL forwards the operation, Nexus executes it and the lock status returns.

However, tuning alone is not playback. Once the frontend is locked, the system must discover and extract useful data from the transport stream. This leads to filtering and table parsing.

PSI/SI Filtering After Tuning

For PSI and SI table parsing, middleware opens filters through the Android Tuner API. Android represents this as opening a filter on the tuner or demux. At the HAL level, the relevant interfaces are demux and filter abstractions. At the Nexus level, this maps to InputBand, ParserBand and PidChannel.

When Android says to set the frontend data source and open a filter, Nexus must configure the corresponding input band, parser band and PID channel. If the filter is intended for PAT or PMT, the PID channel is configured for the corresponding table PID.

Once data is available, the HAL reports a filter event. Middleware then reads the filter output data and parses the PSI or SI section. This connects directly to the broadcast metadata story from earlier lectures. The PAT and PMT are the same tables as before; now we see how software obtains them.

Broadcast A/V Playback Flow

A/V playback introduces MediaCodec and AudioTrack.

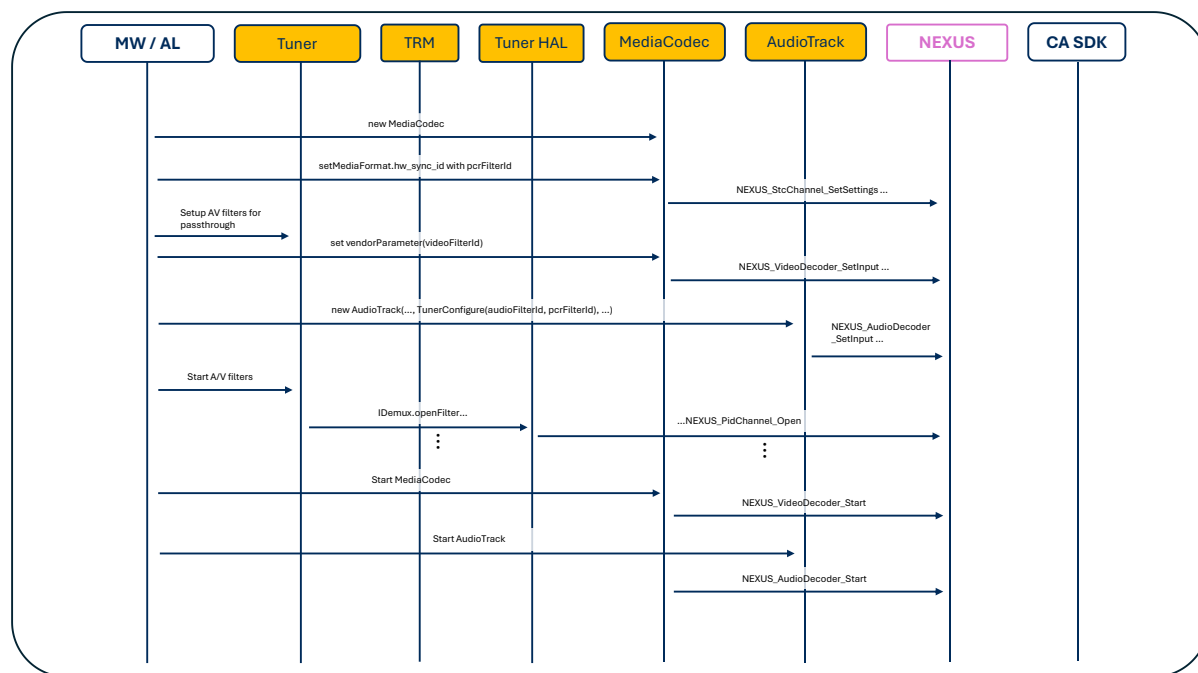


Figure 5. Broadcast A/V Playback UML Sequence

For video playback, middleware creates a MediaCodec instance and configures the media format. In broadcast playback, synchronization is critical. The video and audio streams must share the same timing basis, so the PCR PID or a corresponding hardware synchronization identifier becomes important.

On the Nexus side, this maps to setting up an STC channel. The STC channel is associated with the PCR timing path and allows audio and video decoders to align their PTS values against the same system time clock.

Middleware then configures A/V filters for passthrough or direct playback mode. In normal file or IP playback, MediaCodec receives elementary stream data from software buffers. In broadcast passthrough mode, compressed audio and video data may be routed directly from the tuner/demux pipeline toward hardware decoders, without manually copying all samples through user-space buffers.

Because of this, vendor-specific parameters may be passed to MediaCodec, such as a video filter ID. This tells MediaCodec that the input is not the standard software-fed buffer path, but a tuner/demux filter path. At the Nexus level, this corresponds to connecting the selected PidChannel to the VideoDecoder input.

Audio follows the same idea. AudioTrack is configured with tuner-related synchronization information, and Nexus connects the audio PID channel to the AudioDecoder input. Both audio and video share timing through the STC channel, enabling lipsync.

The goal of all these calls is to construct a Nexus playback graph: InputBand feeds ParserBand; ParserBand produces PID channels; one PID channel goes to VideoDecoder and another to AudioDecoder; the video decoder feeds a VideoWindow on a Display; the display feeds output interfaces; audio goes to audio output interfaces such as DAC or SPDIF. The STC channel connects the timing logic.

This is the transition from a dataflow diagram to software. Earlier, boxes and arrows described the signal path. Now, boxes become objects and arrows become API calls such as open, configure, set input, set output and start.

The system usually prepares the path first and starts processing only afterward. Opening objects and connecting them does not mean data is already flowing. The demux must be configured to filter the A/V PIDs, PID channels must be opened and decoders must be started. On Android, this corresponds to starting MediaCodec and AudioTrack. On Nexus, it triggers video and audio decoder start operations.

The diagram stops at the Nexus level. Beneath Nexus, deeper activity still exists: register writes, demux configuration, decoder setup, buffer management and interrupt handling. For this lecture, the vendor SDK level is sufficiently deep.

Table Parsing Flow

Table parsing is the first real step after tuning. Even after the tuner is locked, middleware still does not know what services exist or which PIDs belong to video, audio and metadata. That information comes from PSI/SI tables such as PAT and PMT.

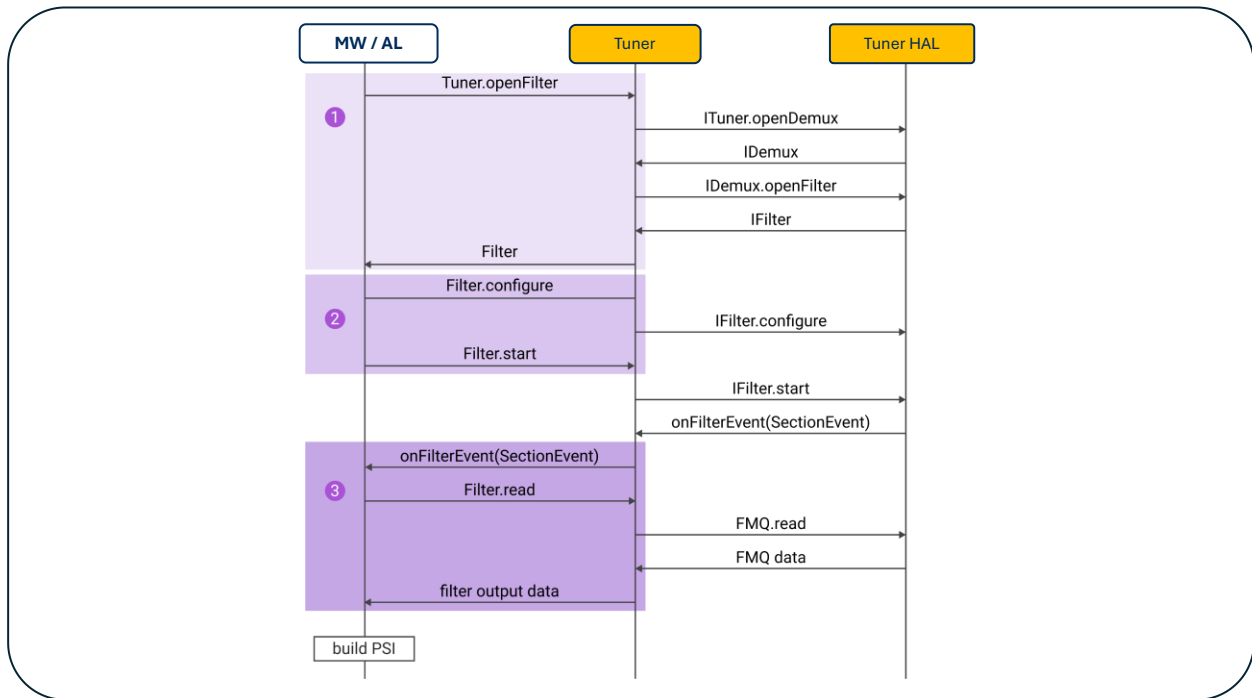


Figure 6. Broadcast table parsing

The flow starts with middleware calling `Tuner.openFilter`. This is an abstraction: middleware asks for a filter without dealing directly with demux hardware. Internally, Android opens a demux via `ITuner.openDemux`, creates a filter via `IDemux.openFilter` and returns a `Filter` object to middleware.

Middleware then calls `Filter.configure`, where it sets the PID and data type it wants. For PAT, this would be PID 0. This is translated to `IFilter.configure` in HAL and then to vendor-specific configuration. In Nexus terms, this corresponds to configuring a `PidChannel` on top of a `ParserBand`.

After that, middleware calls `Filter.start`. Filtering is now active, but data is not returned immediately. Broadcast tables are cyclic, so the system waits until matching section data appears in the transport stream.

When data arrives, the HAL emits `onFilterEvent(SectionEvent)`, which propagates upward as a middleware callback. The event only signals that data is ready; the actual bytes still need to be read.

Middleware then calls `Filter.read`, which internally reads from an FMQ, or Fast Message Queue. FMQ is an efficient shared buffer between HAL and framework. The returned bytes are then parsed by middleware into PAT, PMT or other table structures.

In short, the table parsing flow is: open filter, configure PID, start, wait for event, read data and parse the table.

Memory-Mode Video Playback

A similar producer-consumer pattern appears in memory-mode video playback. A media event tells the upper layer that data is available. Middleware or the application obtains a queue request from MediaCodec, attaches the linear block of data received from the tuner filter, queues it to the codec and later releases the handle when the codec has consumed it.

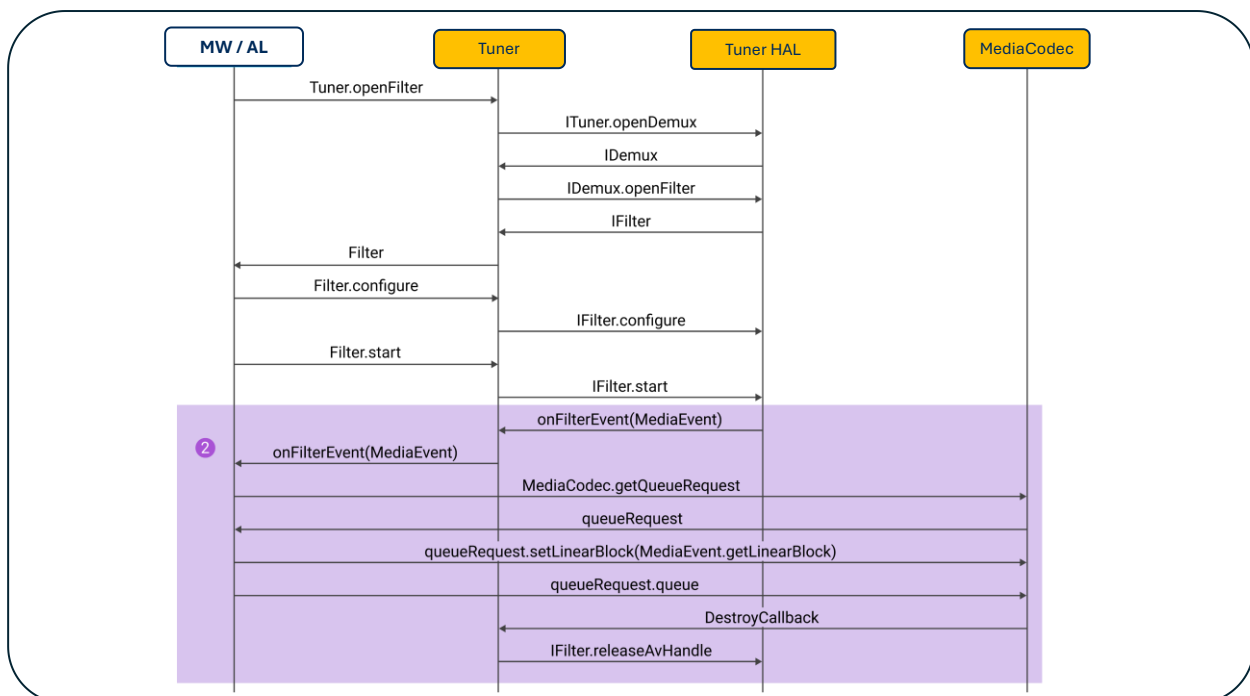


Figure 7. Memory-mode video playback

Whether the system parses PSI sections or feeds compressed A/V data, the recurring pattern is the same: configure the pipeline, start the producer, receive an event, read or queue data and then release or acknowledge the consumed buffer.

This pattern is important for Project 2. Students should avoid drawing only a simplified call such as “middleware calls tuner.” Instead, the diagram should show which object is opened, which resource is requested, which HAL interface is used, which vendor SDK abstraction is reached and where the data or event comes back.

That is how a static stack becomes a real software sequence.

Preparing for Project 2

As students learn more details, their UML sequence diagrams can become richer and more accurate. The diagrams shown in the lecture should not be treated as fixed templates. In a concrete implementation, lifelines may differ, some layers may be combined and additional abstractions may appear depending on the use case.

What matters is not copying the exact diagrams. What matters is understanding the flow and mapping it to the chosen architecture.

The only major part still missing is the middleware stack itself, which is covered in the next lecture. Once middleware is explained, the complete picture of the system will be available: hardware abstractions, platform abstractions, SDKs, middleware and applications.

Project 2 will use this knowledge to move from high-level use cases to software-stack diagrams and sequence diagrams, and later to concrete code.

Closing Perspective

This lecture connected HAL and PAL through real media-device examples. HAL exposes what hardware can do. PAL and SDKs expose what developers need in order to build use cases. Nexus showed how a vendor DTV SDK turns transport streams, PIDs, decoders and display outputs into programmable objects and filter graphs. Android showed how a modern multi-application platform wraps vendor functionality through Tuner SDK, Tuner HAL, TRM, MediaCodec, AudioTrack and Media CAS.

The central idea is that every media use case eventually becomes a configured processing graph. Middleware expresses the user-level goal, the platform translates it into API calls, the DTV SDK builds the vendor-side graph, and the HAL and hardware execute the actual signal processing.

Understanding this mapping is the key step toward making the architecture real in software.